
PUBLIC BUILD ARTIFACT

Agentic Fleet Build Guide

A step-by-step construction manual for one dependable cloud AI employee: its job contract, files, runtime, model policy, Slack review, observability, recovery, and path to a governed fleet.

BUILD ONE REAL JOB. MAKE ITS FAILURE VISIBLE. THEN EARN THE NEXT ROLE.

24-page implementation guide

Roman Bediner | romanbediner.com/resources/agentic-ai-employees

Build in order. Stop after the first useful proof.

This is not a mandate to build a fleet in one sprint. Pages 3 through 19 get one agent to a controlled production pilot. The remaining pages make the system repeatable.

01 **Frame the job** **pages 3-6**

02 **Create the build contract** **pages 7-10**

03 **Run it in the cloud** **pages 11-15**

04 **Make it reliable** **pages 16-19**

05 **Scale with control** **pages 20-24**

Your stopping rule

Do not add a second agent until a real person has received the first output, corrected it if needed, and you can explain the last successful run and the last failed run from evidence.

Choose a job, not a job title.

A first agent succeeds because its output and authority are deliberately small. A vague role such as 'CMO agent' cannot be tested. A repeated outcome can.

JOB

One bounded result

Example: turn approved cloud inputs into one Morning Market Brief by 9:00 AM on weekdays.

RECEIVER

One human moment

A named operator reads the draft in Slack or email, then approves, rejects, or asks for a revision.

LIMIT

One authority boundary

The agent may draft and summarize. It may not publish, spend money, alter source data, or impersonate a human.

PROOF

One evidence record

Every run leaves a run ID, input references, output location, reviewer state, and failure reason if it stops.

Fill this in before writing code

- Trigger: [schedule or event]
- Allowed inputs: [specific cloud sources]
- Output: [format, recipient, deadline]
- Human owner: [role that approves or corrects]
- Failure owner: [role that receives a loud alert]

Use a Morning Market Brief to make the pattern concrete.

The example is intentionally ordinary. It proves the complete loop with low-risk inputs and a human approval point. Substitute your own job after you understand the contracts.

PART	CONCRETE EXAMPLE	DEFINITION OF DONE
Trigger	Weekdays at 8:30 AM	One run begins once. Duplicate triggers reuse the run ID.
Input	Approved cloud folder plus one market-data API	Each source is logged with timestamp and version or URL.
Output	Five-bullet brief posted as a Slack draft	The owner can approve, reject, or request a revision.
Write	Approved brief stored in a durable record	A retry cannot post or store a second final brief.

The build sequence

First make a deterministic draft from sample inputs. Next add the model only where it improves synthesis. Then add the Slack approval action. Last, break it on purpose and prove the alert and retry behavior.

Do not start with more agents. Start with a traceable result.

Define done before implementation makes it feel done.

A useful agent has a testable user outcome, a visible limit, a durable record, and a failure path. This table is the minimum release contract for your first pilot.

QUESTION	A PASSING ANSWER
Useful?	The reviewer can identify a good result in under one minute and correct a bad one.
Bounded?	Allowed data, tool actions, spending, and approval requirements are written down.
Recoverable?	A timed-out or duplicate run stops safely and names an owner and next action.
Observable?	An operator can see the last run, its inputs, output, cost, and review state.
Maintainable?	The PRD, code, tests, deployment instruction, and rollback instruction agree.
Release gate If any answer is unknown, it is a named backlog item, not an assumption. Do not call the system autonomous when it cannot show a human what it did or what happened when it failed.	

Write the one-page build contract before you create the agent.

The PRD is the source of truth for user value and safety. Update it whenever behavior, infrastructure, or a visible promise changes. A prompt alone is not a product requirement.

DOCS/PRD.MD

```
# [AGENT_NAME]
User and outcome: [who receives what, when]
Allowed inputs: [cloud sources and freshness]
Output and review: [format, owner, approve/reject action]
Forbidden actions: [writes, spend, publishing, data limits]
Success: [measurable acceptance criteria]
Failure: [alert owner, retry policy, rollback condition]
```

A PRD review question

Could a new engineer build the first version without inventing product behavior? Could a reviewer reject a change that violates this contract? If not, make the requirement more specific.

Keep implementation details in architecture and runbook files, not hidden in a model prompt.

Give the agent files a human can inspect and change safely.

Personality and skill files should be readable behavior contracts. The runtime should be thin, deterministic where possible, and wired to durable state. Put the knowledge where a reviewer expects it.

RECOMMENDED REPOSITORY

docs/PRD.md	# purpose, authority, acceptance
docs/architecture.md	# components and data flow
docs/runbook.md	# operate, alert, rollback
.agent/personality.md	# voice, judgment boundaries
.agent/skills/brief.md	# exact job instructions
.agent/evals/brief-cases.md	# expected and failing cases
src/run.ts	# orchestration entry point
src/tools/	# narrow, typed external clients
src/state/	# run ledger and idempotency
tests/	# unit, integration, failure tests

Personality file: what belongs there

- Role voice and audience expectation
- Evidence standard: distinguish source fact, inference, and unknown
- Authority boundary: draft only unless a separate reviewed tool grants action
- Escalation behavior: ask rather than guess when source, scope, or confidence is inadequate

Make the job instruction executable, reviewable, and testable.

A skill file is a compact procedure. It says what the agent receives, what it must do in order, what it may not do, and how it demonstrates a successful result.

.AGENT/SKILLS/MORNING-MARKET-BRIEF.MD

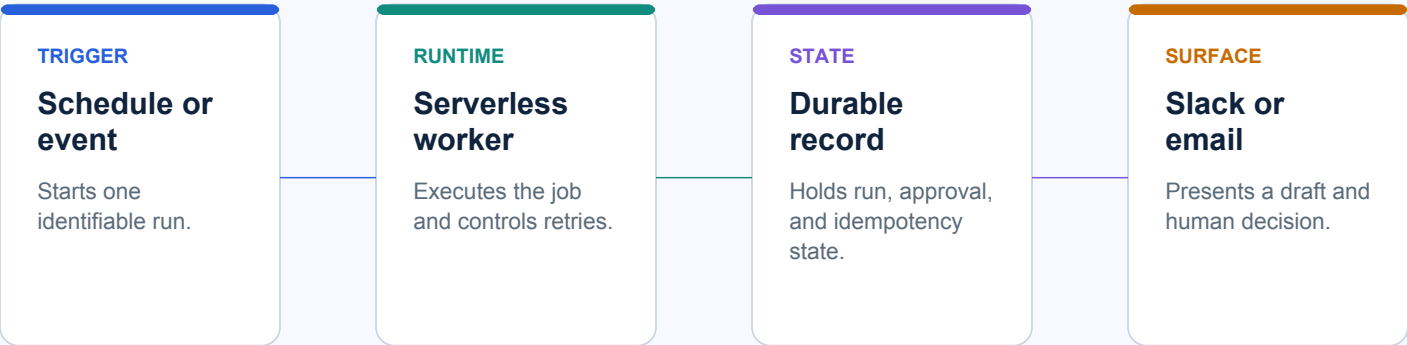
Purpose: produce a five-bullet decision brief.
Inputs: approved folder, approved market-data endpoint.
Steps: validate freshness; extract facts; synthesize; cite sources.
Output: Slack draft with source list and confidence notes.
Never: publish externally, alter a source, infer a missing number.
Escalate: source is stale, incomplete, inaccessible, or contradictory.
Done: owner can approve the draft; run ledger has evidence.

Skill test

Give the skill a representative input, an empty input, a stale input, and a conflicting input. The agent must either produce the stated output or escalate with the correct reason. This is the beginning of an evaluation suite.

Use managed components with one clear responsibility each.

The vendor can change. The contract should not. Keep the first runtime cloud-only so a laptop is a development surface, never the only place an agent can run, remember, or recover.



LAYER	ITS CONTRACT	DO NOT PUT HERE
Behavior	PRD, personality, skills, evaluations	Secrets or silent runtime state
Runtime	Calls tools, enforces timeouts, records outcomes	Business truth that vanishes on restart
State	Run ledger, approvals, memory, dedupe keys	Unreviewed model narration
Surface	Human review and notification	The only copy of the result or audit trail

A trigger starts work. Durable state decides whether work is safe to repeat.

Schedules, webhooks, and queue events are all allowed. None of them are a guarantee that a job starts exactly once. Design each run so duplicate delivery is safe.

1 Create a deterministic run key

Use the job name plus its intended period or source event ID. Store it before doing side effects.

2 Lock or reject a duplicate

If that run key is already in progress or completed, return the existing record instead of starting over.

3 Record every transition

planned -> running -> awaiting_review -> approved or failed. Never infer state from a Slack message alone.

4 Verify the destination

Read back a stored output or inspect an independent success signal before marking a run complete.

MINIMAL RUN RECORD

```
run_id, job_name, trigger_key, status, started_at, ended_at
input_refs[], output_ref, review_state, retry_count, cost_estimate
failure_category, failure_detail, recovery_action, escalation_owner
```

Use service identities, narrow scopes, and a hard stop when access is wrong.

An agent should never quietly borrow a human identity, reach for a broader token, or keep trying an action it is not authorized to perform. Access failure is an observable system event.

NEED	SAFE IMPLEMENTATION	BAD FALLBACK
Read a folder	Dedicated service identity with read-only access to one folder	A founder's personal drive token
Post a review	Slack app restricted to approved channel or direct-message scope	A generic bot that can post anywhere
Call an API	Secret manager reference and scoped API key	A credential copied into a prompt or repository
Write a result	Single durable collection/table with idempotency key	Multiple unaudited writes across chat messages

Fail closed

On missing permission, missing secret, unexpected tool scope, or an unapproved destination: stop the run, preserve the evidence, and alert the owner. Do not impersonate a human, invent a route, or downgrade the restriction.

Select capability by task and risk, not by habit.

Model routing is an operating policy. Start with deterministic logic where it works. Escalate only when ambiguity, synthesis, or impact justifies the additional cost and review.

TASK SHAPE	DEFAULT ROUTE	ESCALATE WHEN
Parse, format, validate	Deterministic code	Rules cannot resolve the input safely
Extract or classify	Lightweight model	Confidence is below threshold or data conflicts
Synthesize a brief	Judgment model	Output could drive a high-impact decision
Approve code or risky action	Independent reviewer plus human gate	Always for production changes and irreversible actions

Route record

- model tier and route reason
- quality or confidence signal
- estimated cost and latency
- whether a human review gate was required

Keep stable instructions stable. Keep live facts outside the cached boundary.

Caching is useful only when it preserves correctness. Treat the prompt prefix as a versioned asset, choose a lifetime that matches the job cadence, and invalidate on policy or source changes.

STABLE PREFIX

Cacheable context

System behavior, personality, skill procedure, approved examples, and static reference material with a version.

DYNAMIC INPUT

Never assume fresh

Current source records, user instructions, time-sensitive facts, approvals, and run-specific evidence.

POLICY

Measure and invalidate

Record hit rate and savings. Expire at the job cadence. Flush on a changed skill, policy, or reference version.

Cache decision checklist

- Is the content byte-stable and safe to reuse across runs?
- Does the TTL end before the content becomes misleading?
- Can the run log identify the prompt or policy version used?
- Will a policy change invalidate the cache automatically or through an explicit operation?

Use the Hivemind pattern: recall, act, verify, capture, fade.

Hivemind is the public name for the shared-memory pattern informing this fleet. It is credited here without linking to external systems. The rule is simple: memory helps work only when its authority and freshness are visible.

RECALL
Recall
Retrieve only role, project, or customer context relevant to this run.

ACT
Act
Use the bounded skill and cite the source records that informed the result.

VERIFY
Verify
Confirm the outcome through human review, evaluation, or an independent success signal.

CAPTURE
Capture
Write a short, scoped lesson with source, date, and confidence.

FADE
Fade
Reduce stale or unconfirmed memory influence instead of silently treating it as truth.

MEMORY FIELD	REQUIRED VALUE
Scope	organization, role, project, or customer boundary
Grounding	source reference, decision date, and confidence
Authority	advisory, reviewed, or superseded
Lifecycle	created, reviewed, corrected, expires or fades

Use Slack to make a human decision easy, not to hide workflow state.

A Slack app can deliver the draft, collect a decision, and expose an escalation. Durable state and evidence live elsewhere. The human should be able to understand the request in one screen.

Example review message

Morning Market Brief is ready for review

Run: MB-2026-08-24 | Inputs: folder 08:20, market API 08:28 | Confidence: normal

1. Competitor launch activity increased in the approved source set. 2. Two items need manual confirmation before any external claim. 3. Full source list is attached to the durable run record.

Approve

Request revision

Escalate issue

Action handler requirements

Authenticate the user, validate the run and current state, write the decision once with an idempotency key, acknowledge the action, and leave a durable audit record. Never trust a button click as the only proof.

If you cannot explain the last run, you do not yet operate the agent.

Use a shared run ID across trigger, tool calls, model route, output, approval, and alert. Structured records make incident triage possible. A dashboard is optional. Evidence is not.

SIGNAL	CAPTURE	OPERATOR QUESTION IT ANSWERS
Trace	run ID and step timing	Where did this run spend time or stop?
Log	structured event and source response	What exactly happened, with what inputs?
Metric	success, latency, cost, cache, corrections	Is quality or economics changing over time?
Record	output, review decision, recovery action	What did the user receive and what is its status?

MINIMUM STRUCTURED EVENT

```
event=run.step_completed run_id=... step=source_validate
job=morning_market_brief status=ok latency_ms=...
input_ref=... output_ref=... route=judgment cache=hit
```

Capture facts. Keep the human-readable explanation in the alert or review surface.

Make every failure loud, classified, and recoverable when safe.

A model response is not success. A run completes only when its output, intended side effect, and evidence record agree. Every mismatch should create a visible owner and next step.

FAILURE	AUTOMATIC ACTION	HUMAN-FACING ALERT
Input missing or stale	Stop before model call	Which source was missing, deadline, owner
Provider timeout	Bounded retry with same run key	Attempts, last error, retry window
Access denied	Fail closed, no fallback identity	Required scope or secret reference, owner
Duplicate delivery	Return existing run record	No alert unless state is inconsistent

A good alert answers four things

- What failed, in plain language
- What evidence is available and where
- What the system already attempted
- Who owns the next decision and by when

Automate repairs that are reversible and known. Escalate changes that alter authority or behavior.

Self-healing is not a license for an agent to rewrite itself in production. Use a documented recovery matrix. Anything that changes permissions, policy, code, prompts, spend, or external impact has a review gate.

CONDITION	SAFE AUTOMATIC RESPONSE	MUST ESCALATE
Transient timeout	Retry with exponential backoff and same idempotency key	Retry budget exhausted or deadline threatened
Stale cache	Invalidate known version and rerun once	Reason for staleness is unknown
Malformed source	Quarantine input and record validation error	Schema or source contract must change
Quality regression	Route to review and open an improvement ticket	Any production prompt or code change
Recovery rule The recovery routine records what it did and verifies its outcome. It never hides a failing condition by silently changing the success definition, bypassing review, or pretending a human approved the result.		

No single agent writes, approves, and ships its own change.

The fleet can surface improvement ideas and prepare a change. Independent review, protected merge, deployment verification, and rollback keep speed accountable. This is where behavior changes become engineering work.

SIGNAL

Signal

A correction, failed run, or recurring friction becomes a specific ticket with evidence.

BUILD

Build

A builder works in a branch with tests, docs, and an explicit rollback condition.

REVIEW

Review

An independent Staff Engineer checks PRD alignment, security, tests, and blast radius.

VERIFY

Verify

After deployment, a health check confirms real behavior or triggers a revert.

Independent authority boundary

The proposer cannot be the only approver. Production access, irreversible changes, new permissions, and policy edits require an independent human gate. Treat a change that fails verification as an incident, not a lesson quietly folded into memory.

Test the job, the boundary, and the failure path before a human depends on it.

An evaluation set does not need to be large to be useful. Start with representative samples and failure cases. Add a case whenever a real reviewer corrects the agent or an incident exposes a gap.

CASE	EXPECTED BEHAVIOR	EVIDENCE
Happy path	Draft uses allowed inputs and meets the output format	Approval-ready output plus source references
Empty input	Stops before synthesis and names the missing source	Failure event and owner alert
Stale input	Marks freshness issue and requests a decision	Timestamp and escalation
Provider outage	Retries safely, then fails loudly	Retry record and alert
Duplicate event	Does not produce a second write or post	Same run record is returned
Rejected output	Captures correction as reviewed feedback, not silent truth	Review decision and backlog item

Keep evaluation cases in version control beside the skill they protect.

Pilot with a small audience, a fixed cadence, and a visible owner.

Your first production period is research, not a victory lap. Watch each run. Gather corrections. Review cost and recovery behavior. Use the evidence to decide whether the job is ready to run more often or for more people.

1 Select a pilot owner

One human receives every output and has authority to stop or revise the pilot.

2 Set a short review window

Run for a stated period with a known cadence. Do not quietly expand scope mid-pilot.

3 Review daily evidence

Check outputs, corrections, failures, retries, latency, cost, and cache behavior.

4 Decide with a scorecard

Keep, change, pause, or scale only after the acceptance criteria are met.

Pilot scorecard

Acceptance rate, correction rate, missed deadline rate, failed-run recovery time, cost per approved result, and whether the owner can explain one successful and one failed run without engineering help.

Name roles by their lane, not by an illusion of unconstrained autonomy.

These are public names and accountability boundaries from the Agentic Society fleet. The private channels, records, routes, schedules, and operating instructions remain private. Add a role only when evidence from the first role creates the need.

ROLE Human Owner Direction and protected decisions	ROLE Director of Fleet Orchestration and Engineering Orchestration and delivery	ROLE Staff Engineer Independent review gate
ROLE Project Manager Work and commitments	ROLE Chief of Staff Knowledge and briefs	ROLE RevOps Engineer Revenue intelligence
ROLE Improvement Engineer Tested learning backlog	ROLE AI Correspondent Editorial intelligence	ROLE Solution Architect Member proposals and design
Fleet boundary No role owns its own production change from proposal through approval and release. Every role has a lane, a memory scope, an observable work record, and a controlled route to human authority.		

Add a role only when you can define its contract with the first role.

The second agent should remove a real constraint. It needs a different job, a different authority boundary, a handoff contract, a memory scope, and a shared way to leave evidence.

BEFORE ADDING A ROLE	ANSWER THIS
Job boundary	What repeated result cannot the first role safely or usefully own?
Handoff	What input does it receive, what output does it return, and who owns a mismatch?
Memory	What may it recall, write, correct, or never see?
Authority	What can it propose, execute, or only ask a human to decide?
Reliability	How will a missing handoff, stale state, or duplicate request fail aloud?
A practical second role If the Morning Market Brief pilot creates recurring corrections, an Improvement Engineer can turn those reviewed corrections into test cases and proposed changes. It does not rewrite the live skill or deploy itself. That division makes the fleet more capable without weakening control.	

Use the Starter Prompt, then keep the files and evidence around it.

The downloadable Agent Builder Starter Prompt gives you a copy-ready first instruction. Fill the bracketed fields, attach it to the PRD and skill contract in this guide, and do not skip the run record, evaluation, or failure path.

STARTER PROMPT OPENING

You are [AGENT_NAME]. Your sole job is [OUTCOME].
You may read only [ALLOWED_SOURCES] and produce [OUTPUT].
You must leave a run record with source references and status.
If input, access, freshness, confidence, or verification is inadequate:
stop safely, preserve evidence, and alert [ESCALATION_OWNER].
Never publish, spend, alter source records, or impersonate a human.

Download the copy-ready Starter Prompt

romanbediner.com/assets/downloads/agent-builder-starter-prompt.md

Use this guide when you need the surrounding operating system: project files, cloud routine, model policy, caching, Slack review, Hivemind memory, observability, self-healing, tests, and independent delivery control.

A reliable agent is not a clever prompt. It is a useful job with evidence and a human boundary.